



Concorrência



Processo

Ambiente de execução auto-contido. Geralmente cada processo tem o seu grupo de recursos alocado, como espaço em memória. Apesar de serem vistos pelo utilizador como um programa, a realidade é que um programa pode ser um conjunto de processos em cooperação. Para facilitar a comunicação entre processos, muitos sistemas operativos suportam Inter Process Communication, recursos como Pipes e Sockets. Estes recursos não são exclusivos a processos dentro da mesma máquina, também podem ser usados entre diferentes sistemas.

A maioria das implementações da JVM correm num só processo. Uma aplicação Java pode criar processos adicionais usando o objeto `ProcessBuilder`.

Thread

Também chamadas de lightweight processes, à semelhança dos processos também fornecem um ambiente de execução. As threads acabam por necessitar de menos recursos para serem criadas e mantidas. As threads existem dentro de um processo, sendo que qualquer processo tem pelo menos uma. As threads partilham os recursos do processo, incluindo memória e ficheiros abertos. Isto torna-as eficientes, mas potencialmente problemáticas se não forem bem geridas.

Execução em multithread é uma feature essencial do Java. Todas as aplicações têm pelo menos uma thread. A thread principal chama-se Main thread e é aquela a partir da qual tudo começa, permitindo criar outras.



Concorrência em Java

`java.util.concurrent` define as features essenciais que suportam alternativas às abordagens integradas à sincronização e comunicação entre threads. Isto inclui:

- Sincronizadores
- Executores
- Coleções concorrentes
- A framework Fork/Join

Os sincronizadores oferecem formas de alto nível de sincronizar interações entre múltiplas threads. As classes de sincronização definidas pela API em `java.util.concurrent` são:

- Semaphore - implementa o clássico semáforo
- CountdownLatch - Espera até que um número específico de eventos tenha acontecido
- CyclicBarrier - Permite que um grupo de threads espere um ponto de execução pré-definido
- Exchanger - Permuta informação entre duas threads
- Phaser - Sincroniza threads que avançam por diversas fases da sua operação



Semáforo

Controla acessos a um recurso partilhado, prevenindo problemas. Caso uma thread queira aceder a um recurso controlado por um semáforo, deve requisitar-lhe uma permissão. Caso o valor do semáforo seja maior que 0, então este dá a permissão e decrementa o seu valor. Caso seja 0, a thread bloqueia até que o acesso seja permitido. Assim que a thread deixa de precisar do recurso, deverá libertar a permissão, fazendo com que o semáforo incremente o seu valor. Se outra thread estiver à espera de permissão, é neste momento que a obtém. A classe Semaphore implementa este mecanismo. Os construtores são os seguintes:

```
Semaphore(int num)  
Semaphore(int num, boolean how)
```

num é o número de permissões inicial. Se for 1, então apenas uma thread poderá aceder ao recurso especificado em simultâneo. Por defeito, caso haja várias threads em espera, a permissão é dada sem ordem específica. Se o booleano how for passado a true, o programador assegura-se de que a ordem de chegada é respeitada na atribuição de permissões. Para obtermos permissão, deveremos fazer uma chamada ao método:

```
void acquire( ) throws InterruptedException  
void acquire(int num) throws InterruptedException
```

A primeira chamada solicita 1 permissão. O segundo faz o pedido para num permissões. Para libertar permissões chamamos:

```
void release( )  
void release(int num)
```



CountDownLatch

Por vezes queremos que uma thread espere até que um ou mais eventos ocorram. Para tal, a API concorrente disponibiliza o `CountDownLatch`. Um `CountDownLatch` é criado com uma contagem de um número de eventos que deverão ocorrer até que a fechadura (latch) seja aberta. De cada vez que um evento ocorre, a contagem é decrementada. Ao chegar a zero, abre. Podemos construir um `CountDownLatch` com o seguinte:

```
CountDownLatch(int num)
```

Num especifica o número de eventos que deverão ocorrer para que a fechadura abra. Para esperar pela abertura, uma thread deverá chamar `await()` que pode ser usado das seguintes formas:

```
void await( ) throws InterruptedException  
boolean await(long wait, TimeUnit tu) throws InterruptedException
```

A primeira forma espera até a contagem associada ao `CountDownLatch` chegue a zero. A segunda aguarda apenas o tempo especificado por `wait`. As unidades são especificadas por `tu`. Retorna `false` caso o limite tenha sido alcançado e `true` se a contagem chegar a 0. Para sinalizar um evento, chamamos `countDown()`. Cada chamada a `countDown()` decrementa a contagem associada.



Cyclic Barrier

Uma situação comum em programação concorrente é quando duas ou mais threads precisam de esperar num certo ponto da sua execução até que todas outras threads tenham chegado a esse ponto. Para podermos lidar com essa situação foi criada a classe `cyclicBarrier`. Permite definir um objeto de sincronização que suspende até que o número especificado de threads tenham chegado a esse ponto.

`CyclicBarrier(int numThreads)`

`CyclicBarrier(int numThreads, Runnable action)`

`numThreads` especifica o número de threads que deverá chegar a esta barreira para que a execução continue. `action` especifica a thread que será executada assim que a condição seja alcançada.

Assim que cada thread chegar à barreira, deve chamar `await()`. Isto pára a execução até que as outras threads cheguem ao seu `await()`. Assim que o número suficiente de threads chegar a este ponto, o `await` é retornado e a execução retomada. É neste ponto que `action` é executado, caso tenha sido declarado um.



Exchanger

O Exchanger simplifica a comunicação entre duas threads. A sua operação é bastante simples: espera que duas threads chamem `exchange()`. Quando isso é feito, troca os dados fornecidos por ambas. Devido à sua simplicidade, torna-se uma forma de comunicação elegante e de uso bastante fácil. A classe Exchanger é genérica:

`Exchanger<V>`

`V` especifica o tipo de dados que está a ser comunicado. O único método desta classe é `exchange()`:

`V exchange(V objRef) throws InterruptedException`

`V exchange(V objRef, long wait, TimeUnit tu) throws InterruptedException, TimeoutException`

`objRef` é uma referência aos dados a serem comunicados. Os dados recebidos pela outra thread são retornados. A segunda forma de `exchange()` permite um temporizador. Tendo em conta que `exchange()` não retorna até que tenha sido chamado no mesmo objeto por duas threads, é uma forma de assegurarmos que uma thread possa continuar caso outra não esteja a responder.



Phaser

O propósito desta classe é permitir a sincronização de threads que representam uma ou mais fases de atividade. Por exemplo, podemos ter um conjunto de threads que implementam 3 fases de um sistema de processamento de encomendas. Na primeira, threads separadas são usadas para validar os dados do cliente, verificar inventário e confirmar os preços. Assim que essa fase termina, a segunda fase tem 2 threads para processar custos de envio e todos os impostos a serem aplicados. Por fim, a última fase confirma o pagamento e determina o tempo estimado de envio. Graças ao Phaser, é possível agora sincronizar facilmente as threads. Esta classe funciona de forma semelhante à `CyclicBarrier`, com a feature adicional de suportar várias fases. Permite definir um objeto de sincronização que espera até uma fase específica tenha terminado. Assim que isto acontece, permite avançar até outro ponto de sincronização.

`Phaser( )`

`Phaser(int numParties)`

`numParties` é o número de registos que o Phaser terá. Para usarmos esta classe, construímo-la e depois usamos `register()` caso não tenhamos fornecido `numParties` no construtor.



Phaser

`int arrive( )`

`int arriveAndAwaitAdvance( )`

`int arriveAndDeregister( )`

`final int getPhase( )`

`arrive()` comunica que uma thread chegou ao fim da sua tarefa ou parte dela. Esta função retorna o número da fase atual. Caso o Phaser já tenha sido terminado, retorna um número negativo. Este método não suspende a execução da thread, continuando sem esperar que a fase esteja toda terminada. Caso, ao contrário do `arrive()` queiramos esperar pelas outras threads, então usamos `arriveAndAwaitAdvance()`.

Uma thread também pode chegar a um ponto de sincronização, ou ao seu fim, e chamar `arriveAndDeregister()`. Isto vai remover o seu registo do Phaser, fazendo com que não volte a esperar por ela.

`getPhase()` permite-nos saber em que fase estamos. Devolve um valor negativo caso a instância tenha sido terminada.



Executor

Esta feature permite iniciar e controlar a execução de threads. Isto significa que, caso tenhamos várias tarefas para correr e apenas duas threads, poderemos correr tudo quase em simultâneo graças à gestão de Executors. Pode ser chamada da seguinte forma:

```
void execute(Runnable thread)
```

O `ExecutorService` expande a área de atuação do Executor. Por exemplo, define `shutdown()`, que pára o o `ExecutorService` invocado. Isto é importante porque se não fizermos `shutdown`, o programa em questão não pode terminar. Também podemos utilizar `ScheduledExecutorService`, que permite “agendar” threads.



Callable

Callable representa uma thread que retorna um valor. Uma aplicação pode usar objetos Callable para computar resultados que serão depois retornados à thread que chamou. Isto facilita a escrita de computações numéricas complexas em que resultados parciais são processados ao mesmo tempo.

```
interface Callable<V>
```

V indica o tipo de dados retornados pela tarefa. Esta classe só tem uma função, `call()`. Dentro dela podemos definir a tarefa que queremos ver executada. Após a tarefa completar, devolve o resultado. Caso este não possa ser computado, `call()` devolve uma exceção. Uma tarefa Callable é executada por um `ExecutorService`, chamando o método `submit()`.

```
Future submit(Callable task)
```

Aqui, `Future` é o que teremos de retorno. Esta é uma interface genérica que representa o valor que irá ser retornado por um objeto Callable. Uma vez que não será obtido no imediato, mas sim no futuro, o nome ajuda a distinguir. Para obter o valor retornado, chamamos `get()` no `Future`. Caso não passemos argumentos, esperamos indefinidamente.

```
V get( ) throws InterruptedException, ExecutionException
```

```
V get(long wait, TimeUnit tu) throws InterruptedException, ExecutionException, TimeoutException
```



TimeUnit

DAYS

HOURS

MINUTES

SECONDS

MICROSECONDS

MILLISECONDS

NANOSECONDS

long convert(long tval, TimeUnit tu)

long toMicros(long tval)

long toMillis(long tval)

long toNanos(long tval)

long toSeconds(long tval)

long toDays(long tval)

long toHours(long tval)

long toMinutes(long tval)



Lock

Locks são fechaduras que trancam um determinado recurso quando várias threads poderão tentar aceder-lhe em simultâneo. Antes de lhe aceder, a thread bloqueia a fechadura, impedindo que outras possam aceder ao recurso. Assim que este acesso estiver completo, liberta a fechadura para outras threads. Se uma thread chegar à Lock e esta estiver bloqueada, esta vai suspender até que a Lock abra. Desta forma podemos assegurar que evitamos alguns dos problemas mais comuns de gestão de recursos. Podemos bloquear/desbloquear uma Lock com as seguintes funções:

`lock()`

`unlock()`

Lock vai aguardar o retorno caso já haja outra thread a bloquear a fechadura. Caso não queiramos que a nossa thread aguarde esse retorno, podemos utilizar a função `tryLock()`, que não espera a disponibilidade do recurso. Nesse caso retorna `true` caso haja outros acessos ao recurso ou `false` caso este esteja disponível. À semelhança de outros elementos de sincronização, também o `tryLock()` permite fornecer um `long wait` para dar um `timeout` caso não consiga o acesso num determinado tempo.



Fork/Join

Uma Framework que simplifica a criação e uso de múltiplas threads. Utiliza vários núcleos do nosso processador de forma automática, para que o programador não tenha de se preocupar com a sua gestão.

ForkJoinTask<V> - Classe abstrata que define uma tarefa (Task)

ForkJoinPool - gere a execução de ForkJoinTasks

RecursiveAction - subclasse de ForkJoinTask<V> para tarefas que não retornam valores

RecursiveTask<V> - subclasse de ForkJoinTask<V> para tarefas que retornam valores

Estas são as classes às quais deveremos fazer extend para tirar partido da framework Fork/Join.



ForkJoinTask<V>

Classe abstrata que define uma tarefa que pode ser gerida por uma ForkJoinPool. O tipo V especifica o tipo de resultado da tarefa. ForkJoinTask distingue-se de uma Thread no sentido em que a ForkJoinTask é uma representação leve de uma Task, em vez de uma thread de execução. As ForkJoinTasks são executadas por threads geridas por uma thread pool de tipo ForkJoinPool. Este mecanismo permite que um grande grupo de tasks seja gerido por um pequeno número de threads. Desta forma, tornam-se bastante eficientes quando comparados a threads.

```
final ForkJoinTask<V> fork( )
```

```
final V join( )
```

O método fork() submete a Task invocada para execução assíncrona. Isto significa que a Thread que chama fork() continua a correr. O método join() espera até a tarefa em que foi chamada termine. O resultado de uma tarefa é retornado. Assim sendo, com o uso destes dois métodos podemos facilmente começar uma ou mais tarefas e esperar que elas terminem.

Outro método importante é o invoke(). Combina o fork e o join numa só chamada uma vez que começa uma tarefa e espera pelo seu fim. Também podemos invocar mais que uma tarefa de uma só vez através do uso de invokeAll(). Este método aguarda o fim de todas as tarefas invocadas.

```
static void invokeAll(ForkJoinTask taskA, ForkJoinTask taskB)
```

```
static void invokeAll(ForkJoinTask ... taskList)
```



RecursiveAction

Se uma tarefa tiver um valor retornado do tipo void, então tipicamente iremos estender esta classe. Apenas um método desta classe tem interesse, `compute()`. O código da `RecursiveAction` a executar deverá estar dentro deste método. Ele representa a parte computacional da tarefa.

```
protected abstract void compute( )
```

Reparemos que é um método `protected` e `abstract`. Isto significa que deverá ser implementado numa subclasse. Em geral, `RecursiveActions` são usados para implementar uma estratégia recursiva, “dividir e conquistar”.



RecursiveTask<V>

Esta classe encapsula uma tarefa que retorna um resultado, especificado por V. Deveremos, mais uma vez, estender esta classe caso queiramos implementar uma tarefa deste género. Tal como nas RecursiveActions, a parte computacional da tarefa deverá estar dentro do método compute(). Também é usada para uma resolução de problemas recursivamente.



ForkJoinPool

A execução de ForkJoinTasks ocorre dentro da ForkJoinPool, que também as gere. Assim sendo, precisamos de ter primeiramente uma ForkJoinPool para poder correr as nossas tarefas. Podemos criá-la explicitamente ou utilizar aquilo a que se chama uma common pool. Isto é uma ForkJoinPool estática que está automaticamente disponível ao programador a partir do JDK 8. Os seguintes são dois dos métodos mais comuns para construir uma ForkJoinPool.

```
ForkJoinPool( )
```

```
ForkJoinPool(int pLevel)
```

pLevel aqui indica o nível de paralelismo, ou seja, o número de threads que podem executar em simultâneo. Tem de ser maior de 0. Isto não indica o número de tarefas que podem ser geridas, essas poderão ser muitas mais. Outro ponto a ter em atenção é que o nível de paralelismo é um objetivo, não é garantido que seja atingido.

Após instanciação, podemos começar as tarefas. A primeira a ser criada é tida como a principal. Normalmente a tarefa principal começa sub-tarefas que também são geridas pela pool. Podemos começar uma tarefa da seguinte forma:

```
T invoke(ForkJoinTask task)
```

Este método aguarda o fim da tarefa para continuar. Caso não queiramos aguardar pelo seu resultado, podemos optar pelo método execute.

```
void execute(ForkJoinTask task)
```

Neste caso, a execução continua de forma assíncrona.



Dividir e conquistar

De forma geral, quando utilizamos a framework Fork/Join estamos a aplicar uma estratégia de dividir e conquistar baseada em recursão. É por isso que as duas subclasses de ForkJoinTask são chamadas de RecursiveAction e RecursiveTask.

Esta estratégia é baseada em recursivamente dividir uma tarefa em subtarefas mais pequenas até o tamanho da subtarefa ser pequeno o suficiente para ser executada sequencialmente. Por exemplo, uma tarefa que aplica uma transformação a cada elemento num array de N inteiros pode ser dividido em duas subtarefas em que cada uma transforma metade dos elementos do array. Por sua vez, também podemos dividir cada uma destas tarefas em outras duas que por sua vez vão executar o código em metade desse array, ou seja, 4 tarefas que executam o código em $\frac{1}{4}$ do array inicial. Podemos continuar esta divisão até cada tarefa executar o código na unidade mais pequena possível, isto é, termos N tarefas para transformar N inteiros, ou até deixar de ser produtivo continuar a dividir a execução em subtarefas e ser mais rápido efetuar as mudanças de forma sequencial.

A vantagem desta estratégia é o facto de o processamento poder ser efetuado de forma paralela. O resultado de um elemento não depende do resultado de outro. Assim sendo, em vez de percorrermos um array inteiro com uma única thread, partes do array podem ser processadas em simultâneo. Naturalmente isto não se aplica apenas a arrays, a maior parte envolve algum tipo de array, coleção ou outro agrupamento de dados.

Um dos pontos-chave para empregar a estratégia de forma eficiente é seleccionar o limite em que processamento sequencial (em vez de continuar a dividir). Tipicamente conseguimos perceber isso através de profiling às características de execução. A documentação da API de Java recomenda que cada tarefa deverá executar entre 100 e 10,000 passos computacionais. Se cada passo computacional for demasiado longo, então talvez seja melhor diminuir o limite de tarefas.